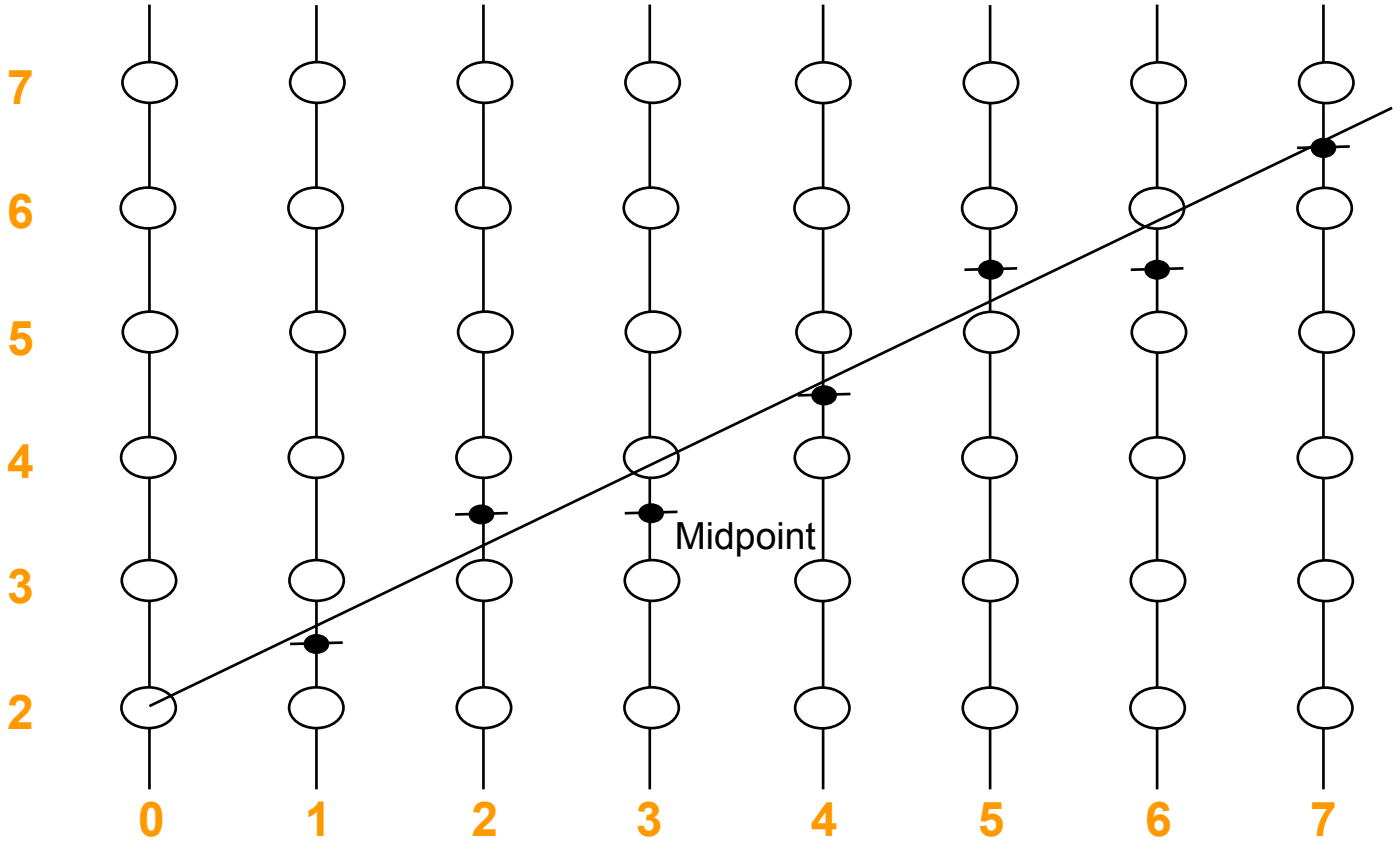
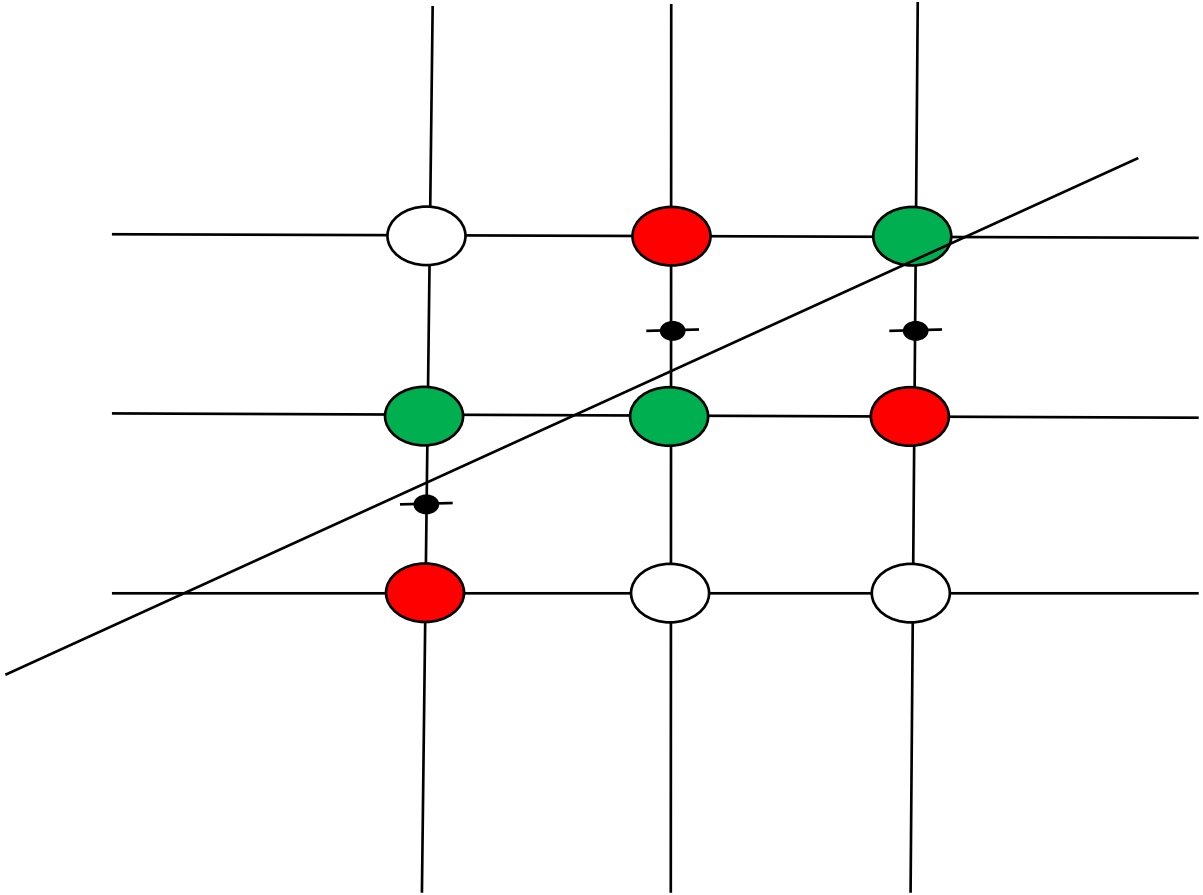
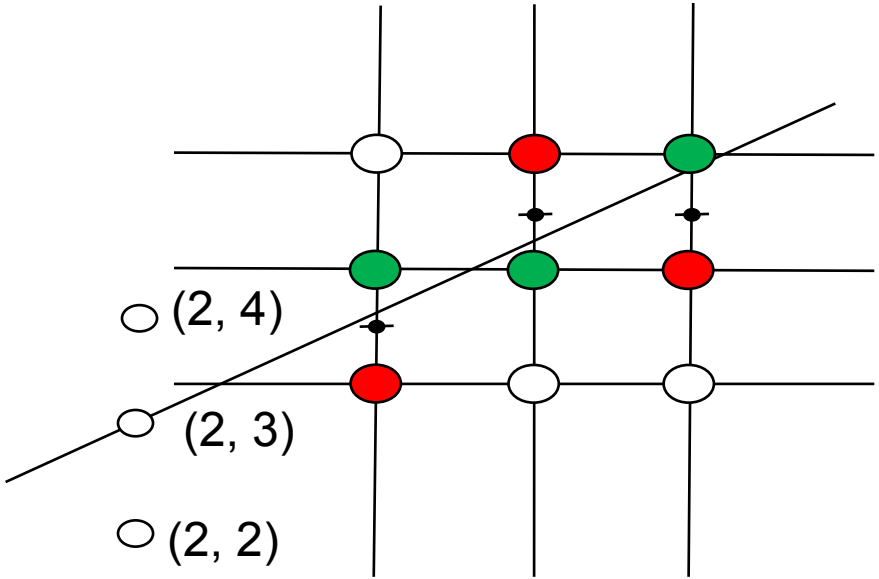
# Computer Graphics: Line Drawing Algorithms

Scan Conversion Algorithms

(Midpoint Line)


7
6
5
4
3
2
0
1
2
3
4
5
6
7
Midpoint



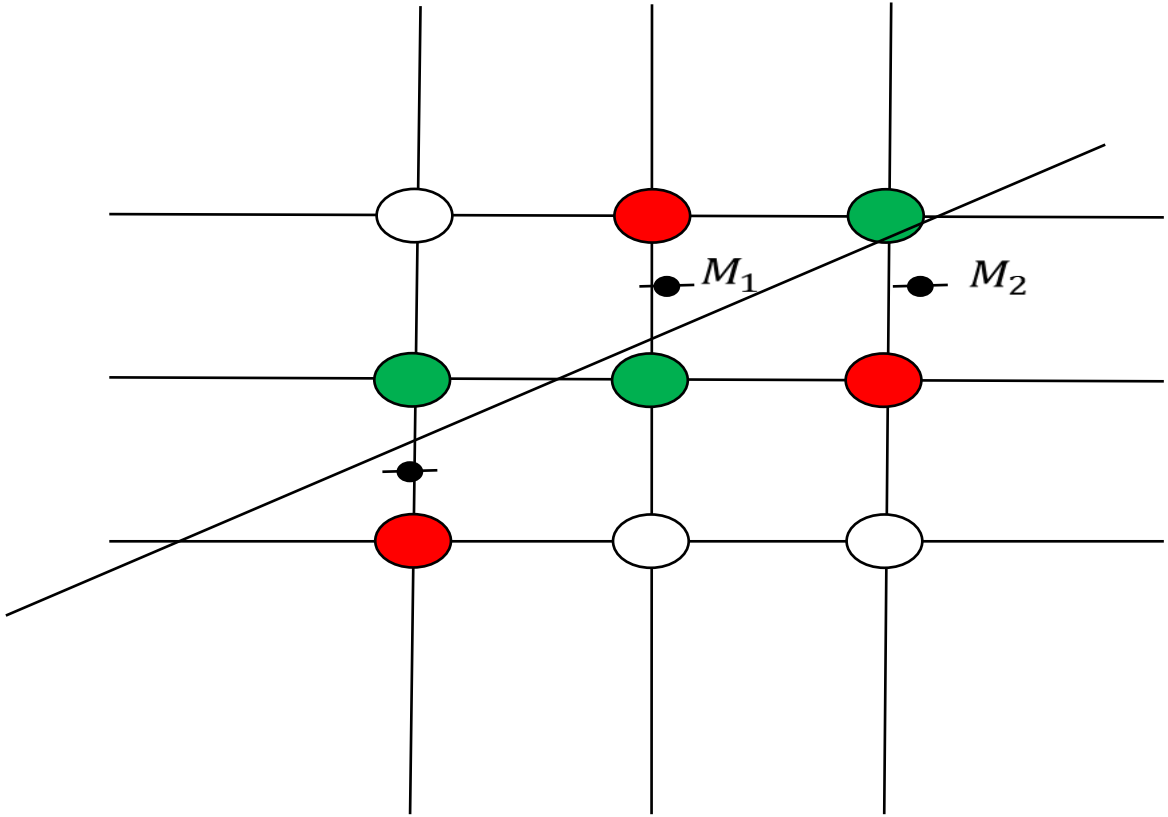


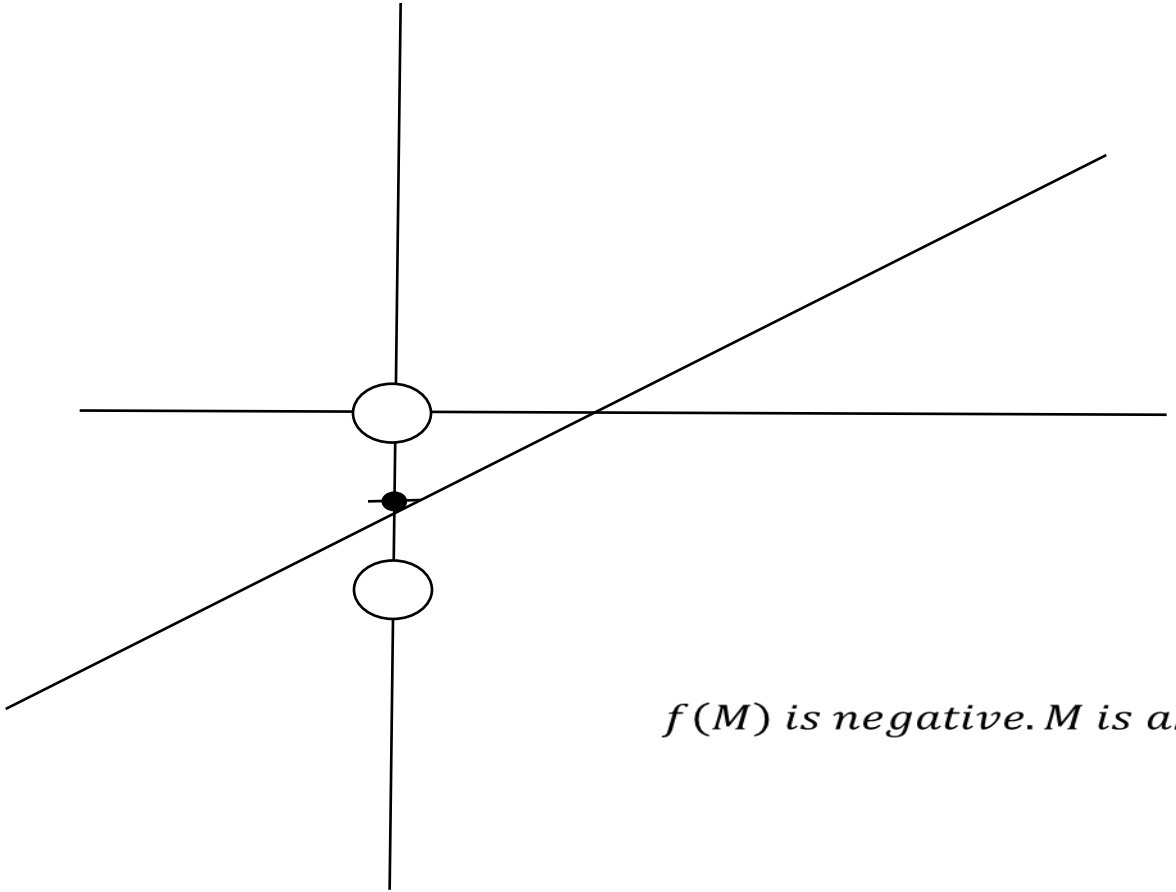

$$ax + by + c = 0$$

$$f(x, y) = ax + by + c$$

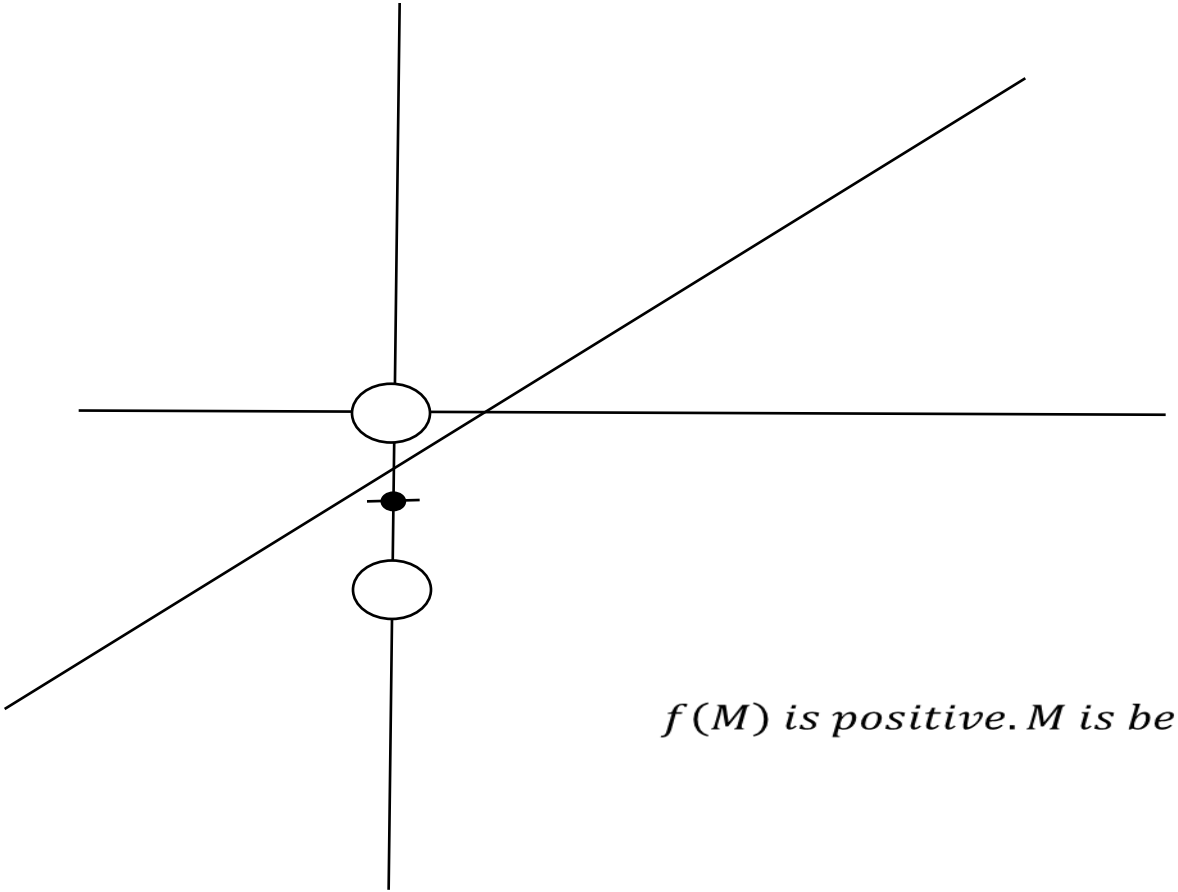
$$f(2, 4) = (-)ve$$

$$f(2, 2) = (+)ve$$


$M_1$
$M_2$

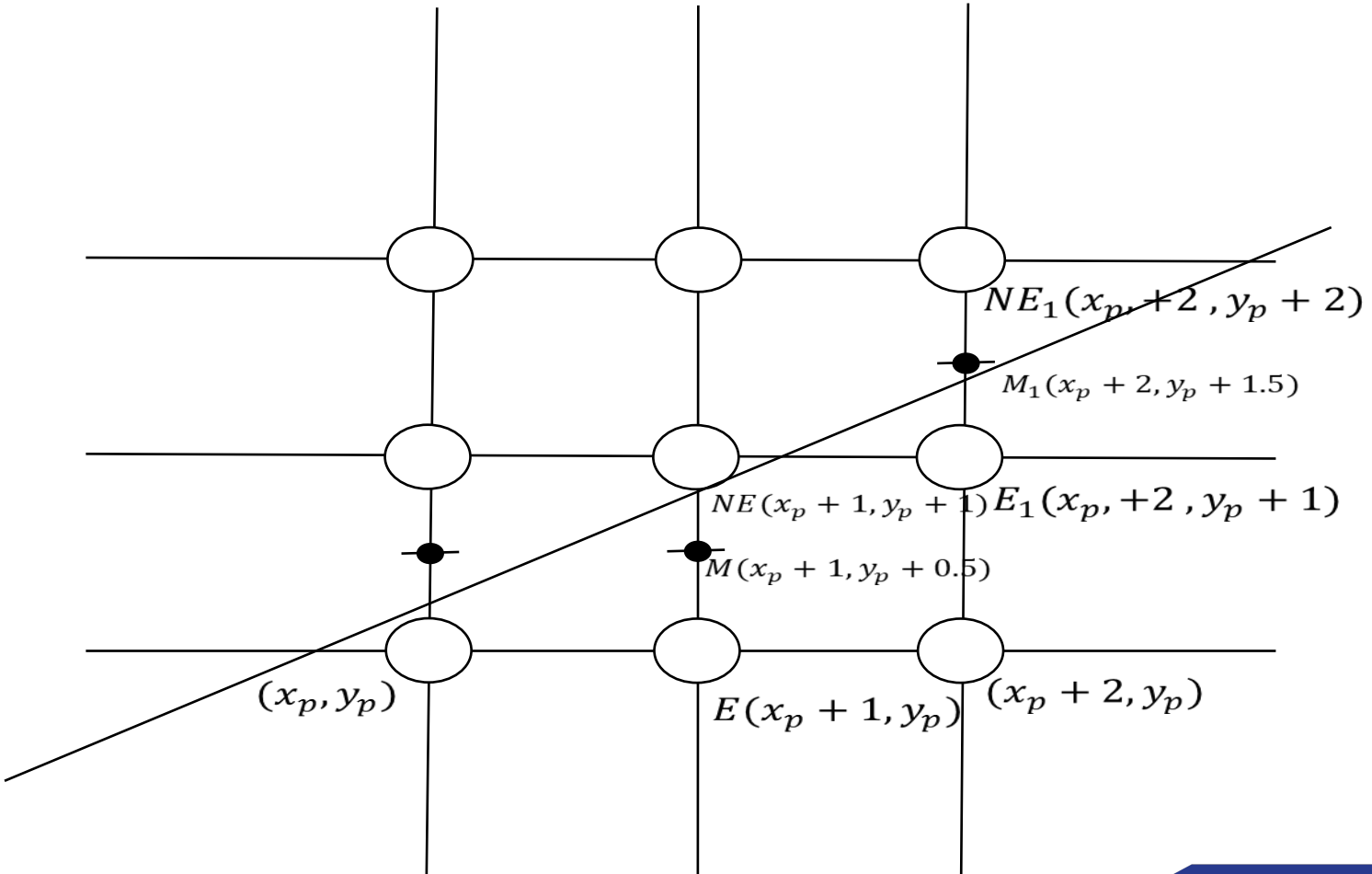


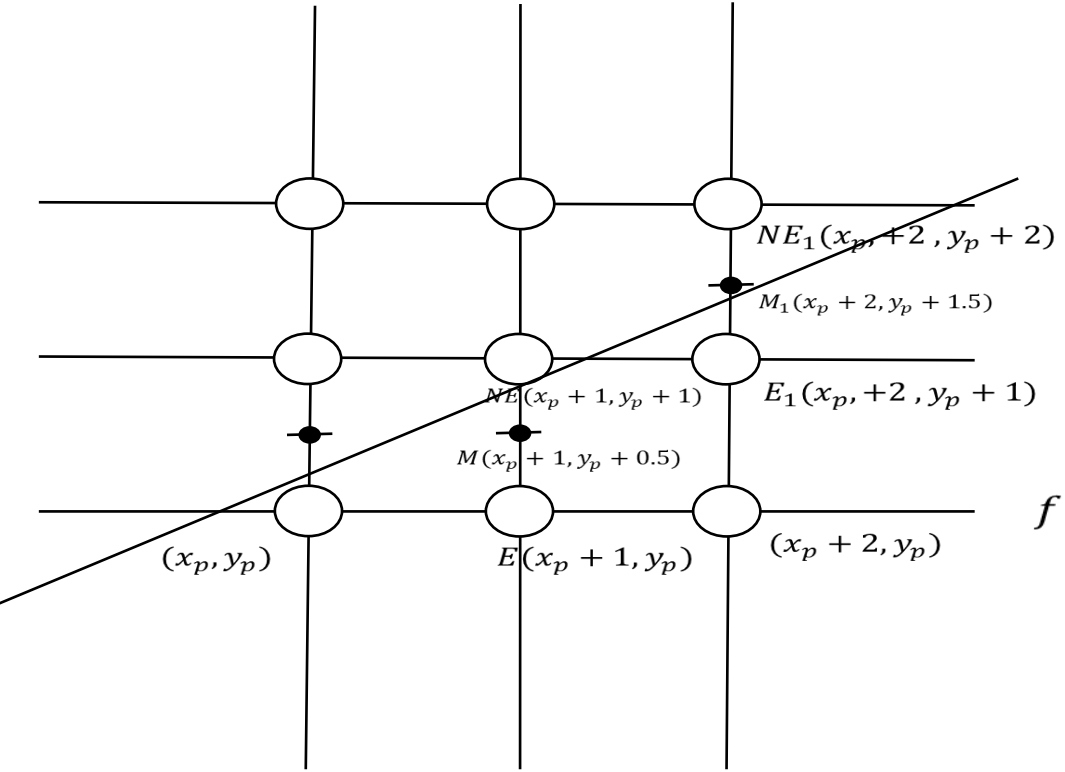
$f(M)$ is negative. $M$ is above the line



$f(M)$ is positive. $M$ is below the line

| f(M) | Pixel chosen |
| --- | --- |
| $f(M) > 0$ | upper (NE) |
| $f(M) \leq 0$ | lower (E) |


$NE_1(x_p, +2, y_p + 2)$
$M_1(x_p + 2, y_p + 1.5)$
$NE(x_p + 1, y_p + 1)$
$E_1(x_p, +2, y_p + 1)$
$M(x_p + 1, y_p + 0.5)$
$(x_p, y_p)$
$E(x_p + 1, y_p)$
$(x_p + 2, y_p)$

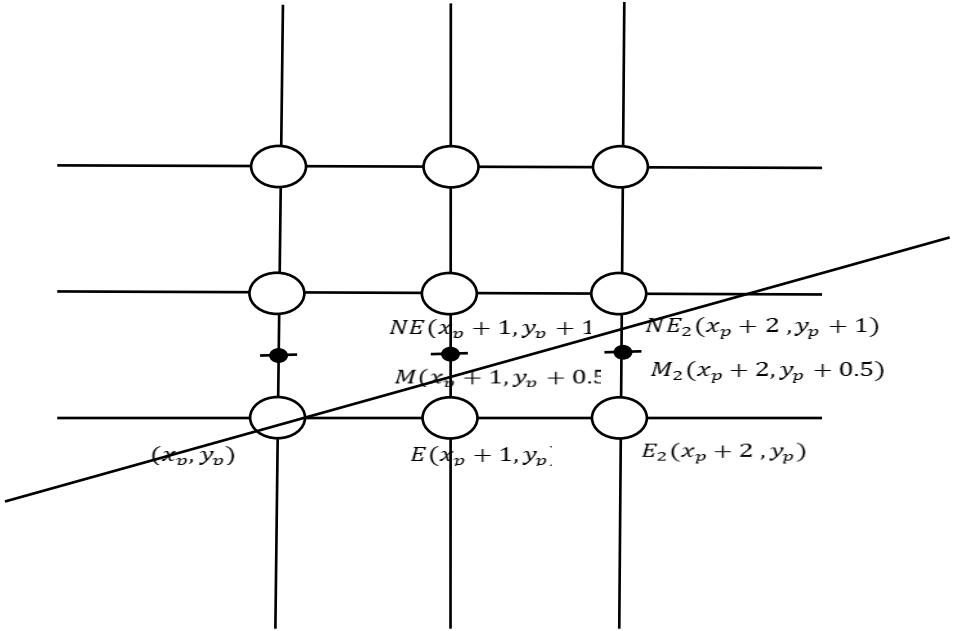



$$f(M) = f(x_p + 1, y_p + 0.5)$$
$$= a(x_p + 1) + b(y_p + 0.5) + c$$
$$d = ax_p + by_p + c + a + 0.5b$$

Suppose, $d > 0$, so $NE$ is chosen

$$f(M_1) = f(x_p + 2, y_p + 1.5)$$
$$= a(x_p + 2) + b(y_p + 1.5) + c$$
$$d_{new} = ax_p + by_p + c + 2a + 1.5b$$

$$d_{new} - d = a + b$$
$$d_{new} = d + a + b$$




$$f(M) = f(x_p + 1, y_p + 0.5)$$
$$= a(x_p + 1) + b(y_p + 0.5) + c$$
$$d = ax_p + by_p + c + a + 0.5b$$

Suppose, $d \leq 0, so\ E\ is\ chosen$

$$f(M_2) = f(x_p + 2, y_p + 0.5)$$
$$= a(x_p + 2) + b(y_p + 0.5) + c$$
$$d_{new} = ax_p + by_p + c + 2a + 0.5b$$

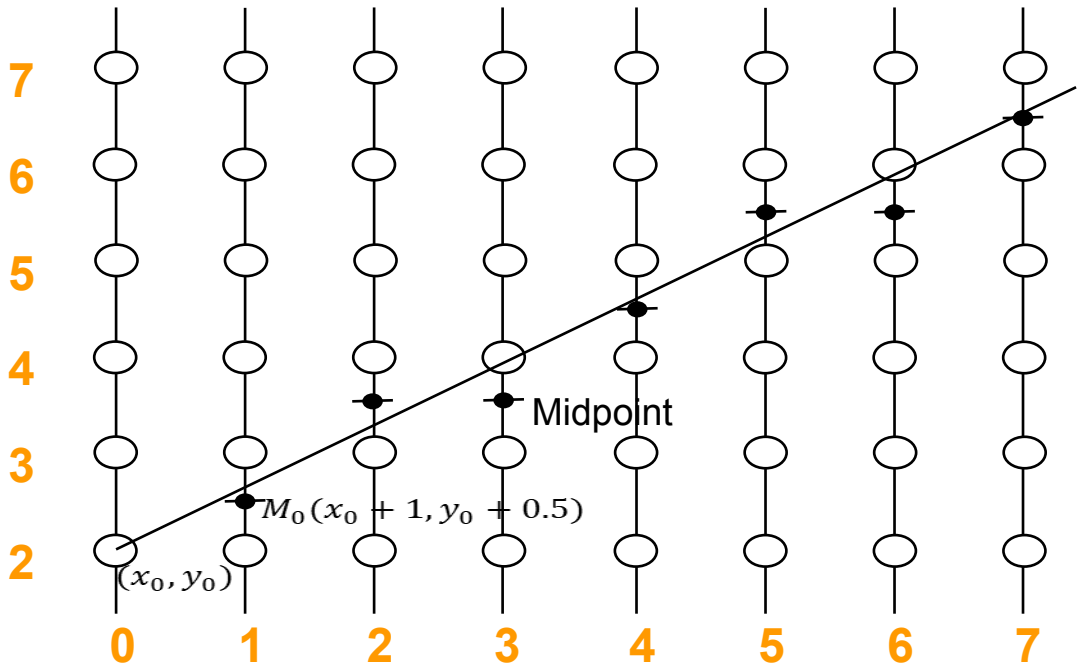
$$d_{new} - d = a$$
$$d_{new} = d + a$$

Calculate d for 1st column.

Choose E/NE.

Update $d_{new}$ acc. to E/NE.

Use $d_{new}$ to choose E/NE again and repeat the loop until the end.



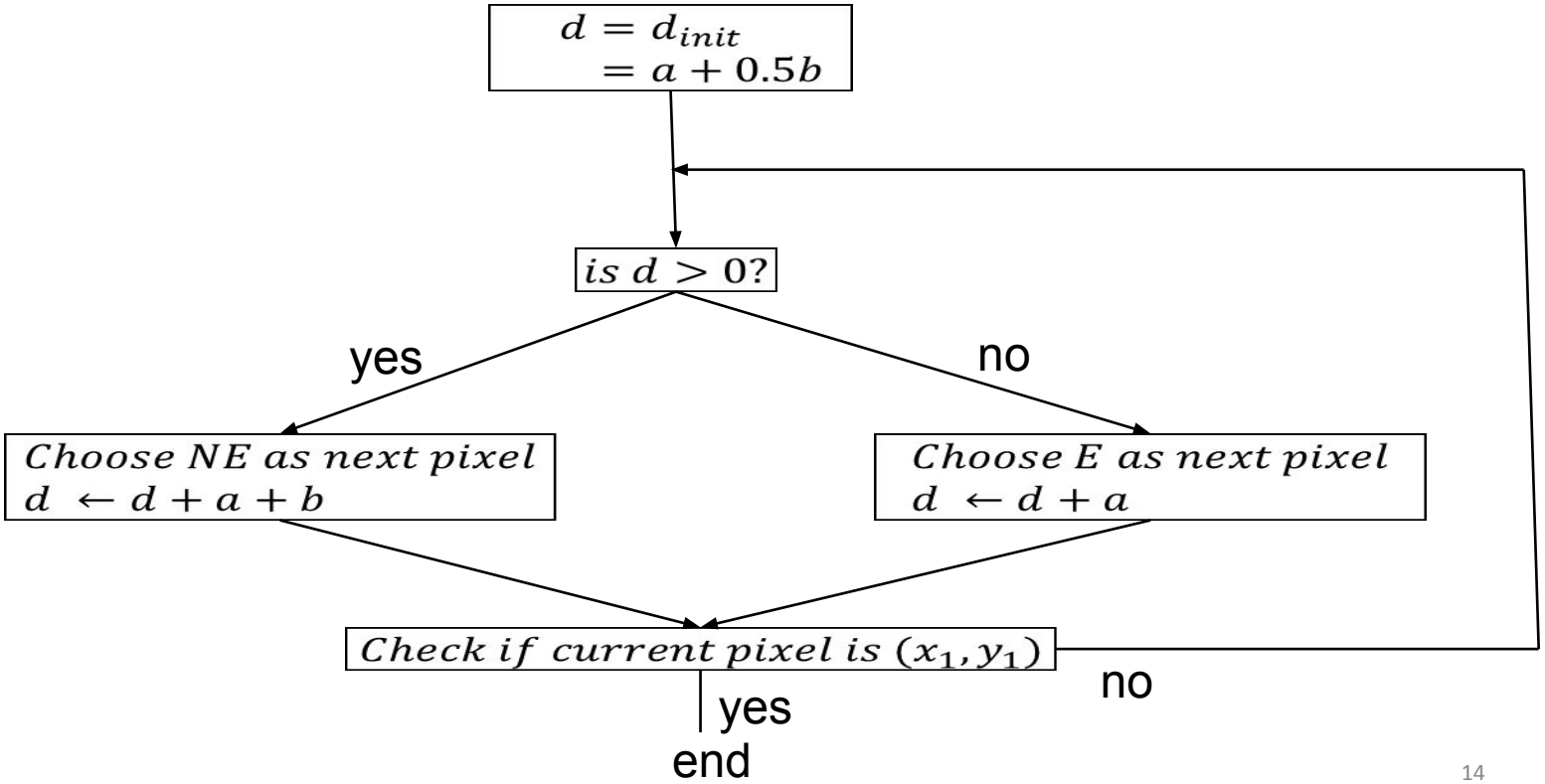

$$
\begin{aligned}
d_{init} &= f(M_0) \\
&= f(x_0 + 1, y_0 + 0.5) \\
&= a(x_0 + 1) + b(y_0 + 0.5) + c \\
&= ax_0 + a + by_0 + 0.5b + c \\
d_{init} &= ax_0 + by_0 + c + a + 0.5b
\end{aligned}
$$

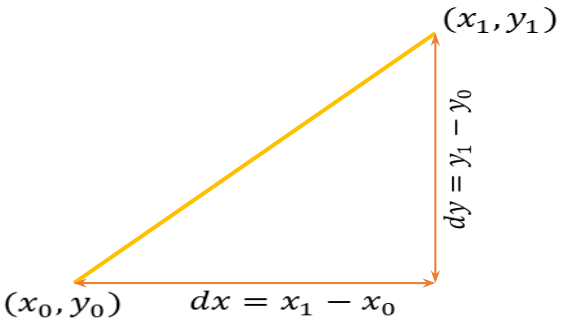
$$ax + by + c = 0$$

$$true\ for\ all\ (x, y)\ on\ the\ line$$

$$So, ax_0 + by_0 + c = 0$$

$$
\begin{aligned}
d_{init} &= ax_0 + by_0 + c + a + 0.5b \\
&= 0 + a + 0.5b \\
&= a + 0.5b
\end{aligned}
$$


$d = d_{init}$
$= a + 0.5b$
$is\ d > 0?$
yes
no
$Choose\ NE\ as\ next\ pixel$
$d \leftarrow d + a + b$
$Choose\ E\ as\ next\ pixel$
$d \leftarrow d + a$
$Check\ if\ current\ pixel\ is\ (x_1, y_1)$
no
yes
end




$$y = mx + B$$

$$m = \frac{dy}{dx} \text{ where } dy = y_1 - y_0 \text{ and } dx = x_1 - x_0$$

$$y = \frac{dy}{dx}.x + B$$

$$y.dx = dy.x + B.dx$$

$$0 = dy.x - y.dx + B.dx$$

$$dy.x - dx.y + B.dx = 0$$

Comparing this with,

$$ax + by + c = 0$$

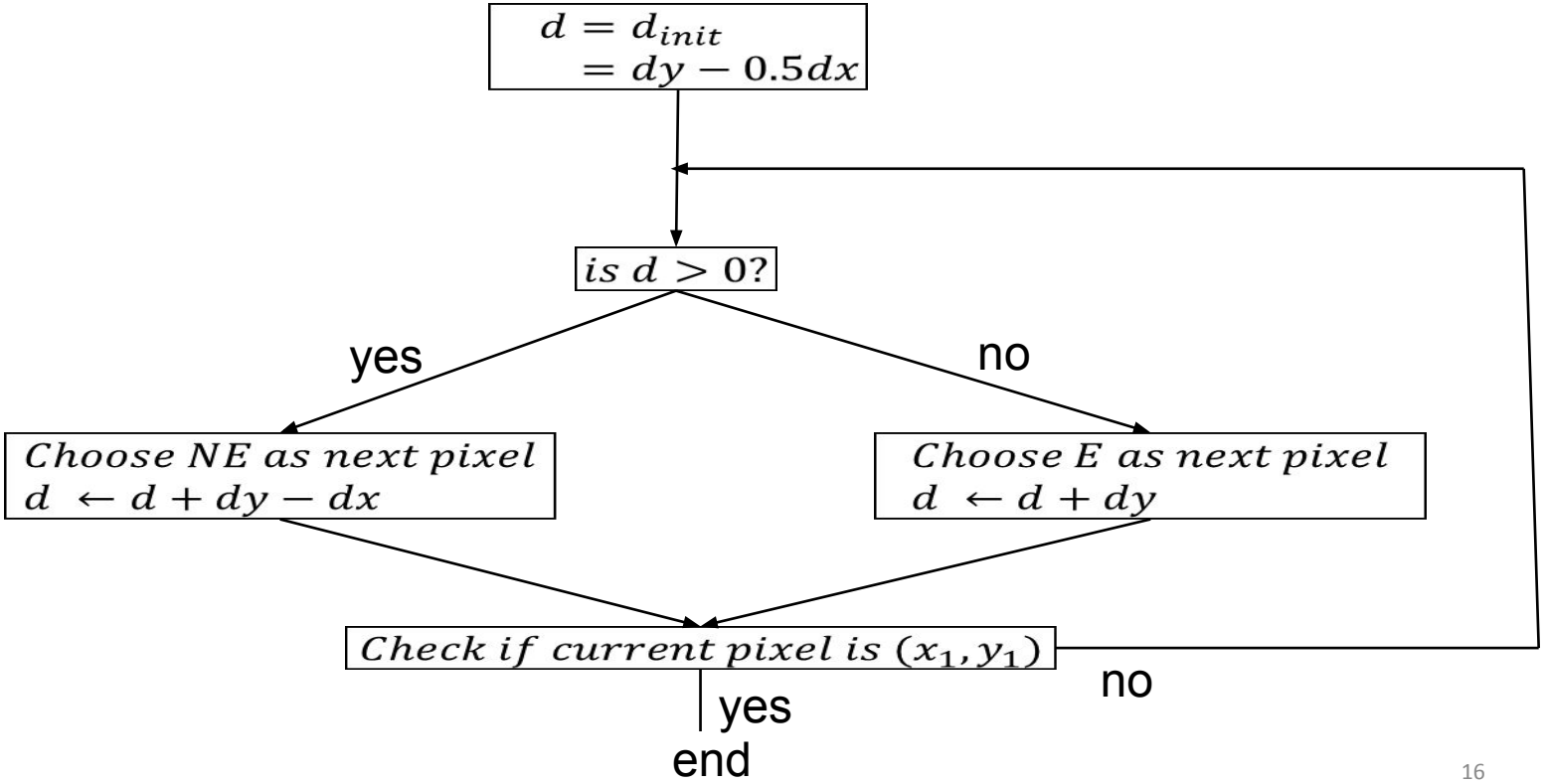
We get,

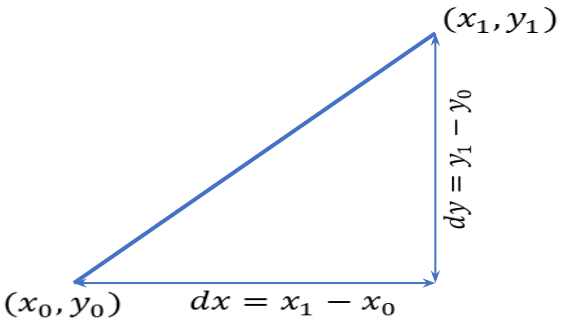
$$a = dy$$

$$b = -dx$$

$$c = B.dx$$

$Putting,$
$a = dy$
$b = -dx$






$$y = mx + b$$

$$m = \frac{dy}{dx} \text{ where } dy = y_1 - y_0 \text{ and } dx = x_1 - x_0$$

$$y = \frac{dy}{dx}.x + b$$

$$y.dx = dy.x + b.dx$$

$$0 = dy.x - y.dx + b.dx$$

$$\boldsymbol{dy.x - dx.y + b.dx = 0}$$

*Comparing this with,*

$$\boldsymbol{ax + by + c = 0}$$

*We get,*

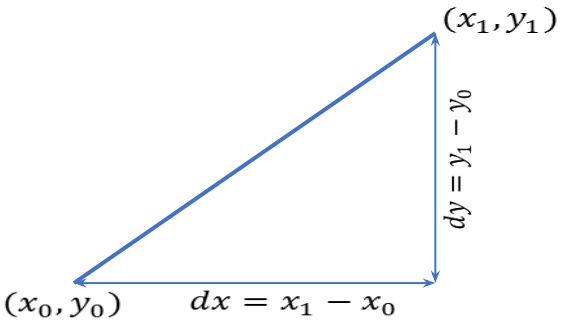
$$a = dy$$

$$b = -dx$$

$$c = b.dx$$

$$2x + 3y + 1 = 0$$

$$4x + 6y + 2 = 0$$




$$y = mx + b$$

$$m = \frac{dy}{dx} \text{ where } dy = y_1 - y_0 \text{ and } dx = x_1 - x_0$$

$$y = \frac{dy}{dx}.x + b$$

$$y.dx = dy.x + b.dx$$

$$0 = dy.x - y.dx + b.dx$$

$$\boldsymbol{2dy.x - 2dx.y + 2b.dx = 0}$$

*Comparing this with,*

$$\boldsymbol{ax + by + c = 0}$$

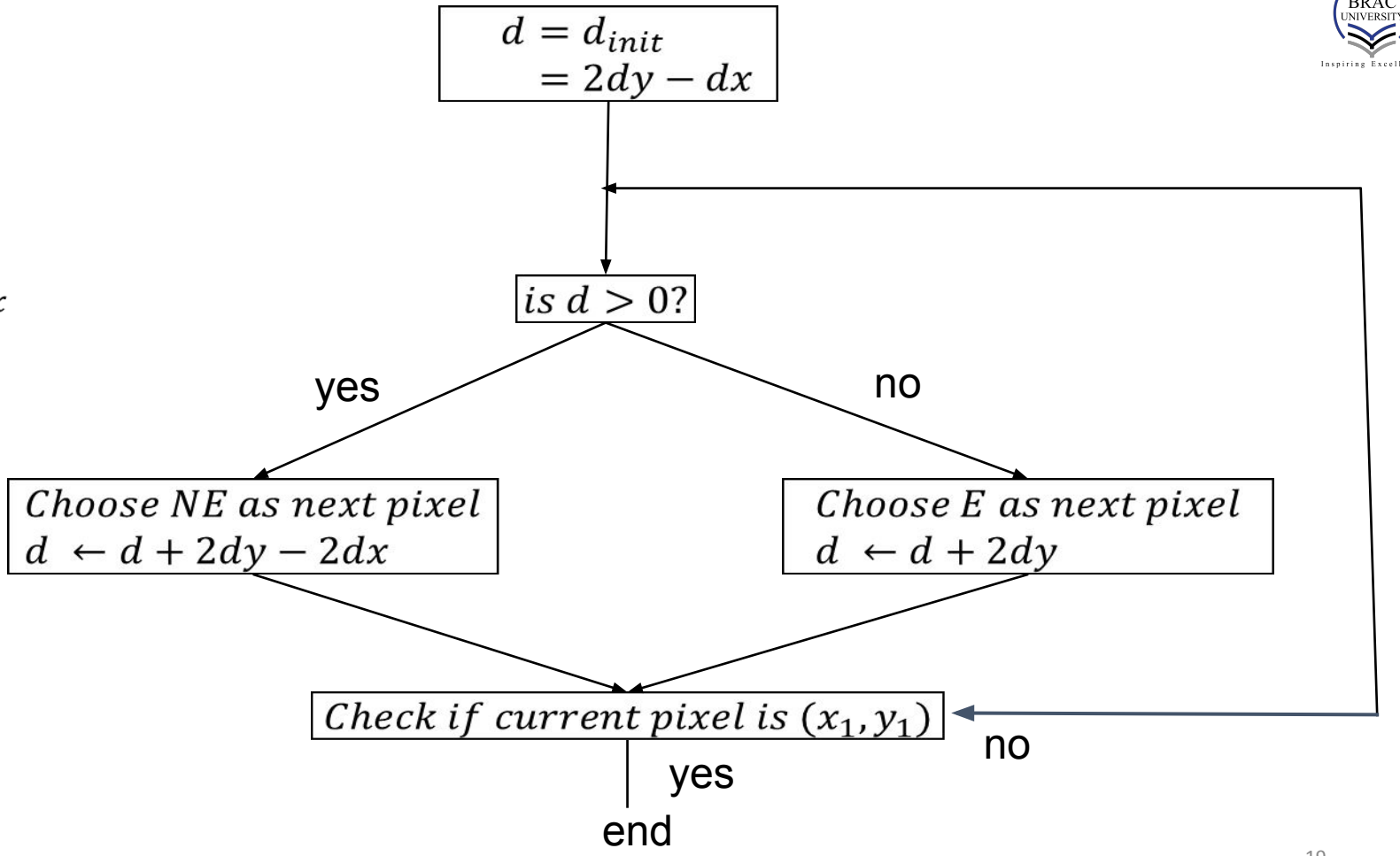
*We get,*

$$a = 2dy$$

$$b = -2dx$$

$$c = 2b.dx$$

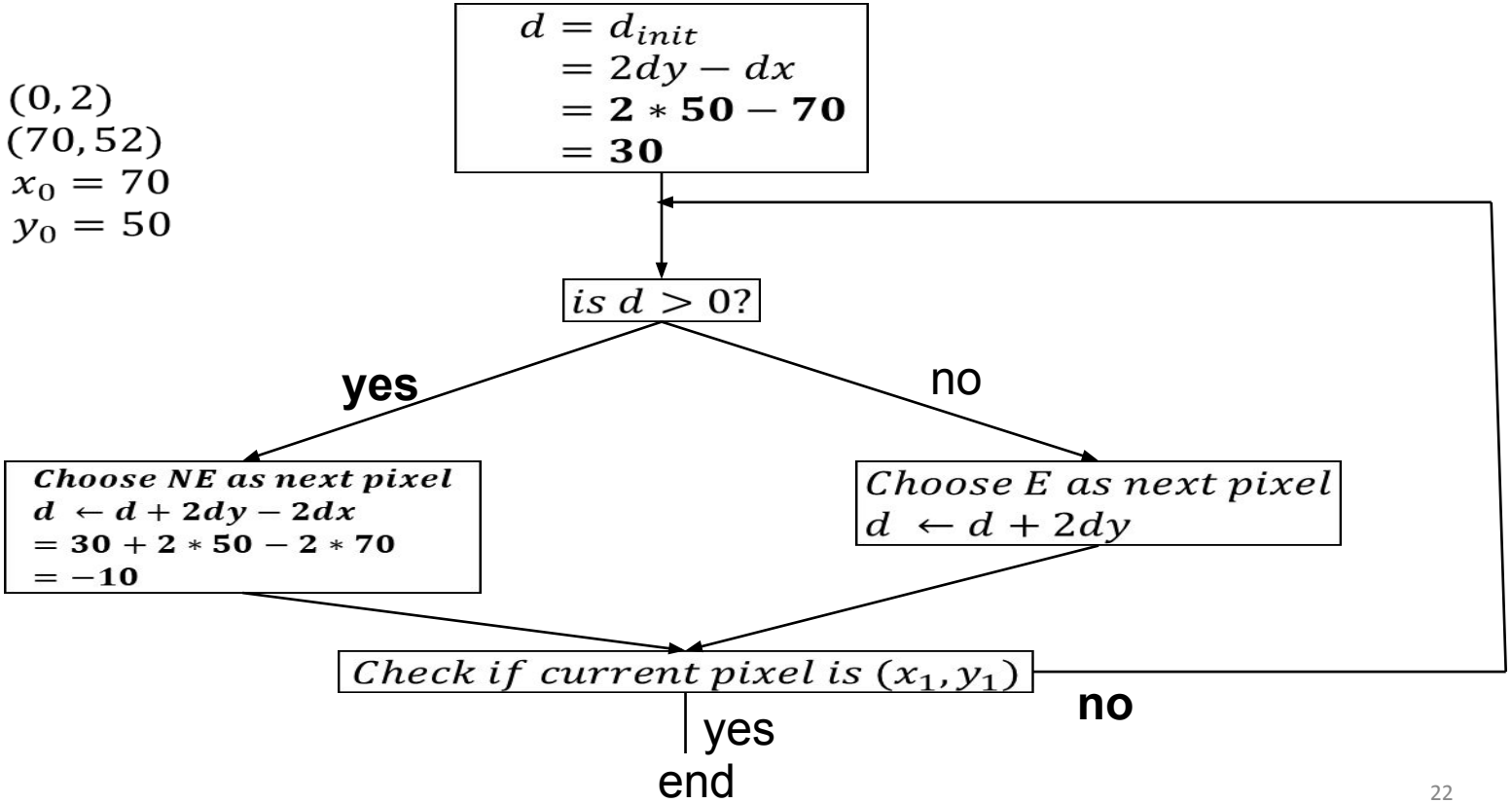
$$Putting,$$
$$a = 2dy$$
$$b = -2dx$$



```
func MidpointLine(int x0, int y0, int x1, int y1, int value){
	int dx, dy, incrE, incrNE, d, x, y;
	dx = x1 -x0;
	dy = y1 -y0;
	d = 2 * dy - dx;
	incrE = 2 * dy;
	incrNE = 2 * (dy - dx);
	x = x0;
	y = y0;
	WritePixel (x, y, value);
	while (x < x1) {
		if (d <= 0) {
			//choose E
			d = d + incrE;
			x = x + 1;
		}
		else {
			//choose NE
			d = d + incrNE;
			x = x + 1;
			y = y + 1;
		}
		WritePixel (x,y, value) //The selected pixel closest to the line
	}
}
```

**Find out the first 7 pixels of the line segment starting from (0, 2) to (70, 52) using midpoint line algorithm**

$(x_0, y_0) = (0, 2)$
$(x_1, y_1) = (70, 52)$
$dx = x_1 - x_0 = 70$
$dy = y_1 - y_0 = 50$



| x | y | d | NE(+1,+1)/E(+1,0) | d updating | Pixel |
|---|---|---|---|---|---|
| 0 | 2 | **30** | **NE** | **30+2x50-2x70 = -10** | **(0, 2)** |
| **1** | **3** | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |
| | | | | | |

$$d_{init} = 2dy - dx = 2.50 - 70 = 30$$
$$\Delta d_{NE} = 2dy - 2dx = 2.50 - 2.70 = -40$$
$$\Delta d_E = 2dy = 2.50 = 100$$

| x | y | d | NE(+1,+1)/ E(+1,0) | d updating | Pixel |
|---|---|---|---|---|---|
| 0 | 2 | 30 | NE | 30+$\Delta d_{NE}$ = -10 | (0, 2) |
| 1 | 3 | -10 | E | -10+$\Delta d_E$ = 90 | (1, 3) |
| 2 | 3 | 90 | NE | 90+$\Delta d_{NE}$ = 50 | (2, 3) |
| 3 | 4 | 50 | NE | 50+$\Delta d_{NE}$ = 10 | (3, 4) |
| 4 | 5 | 10 | NE | 10+$\Delta d_{NE}$ = -30 | (4, 5) |
| 5 | 6 | -30 | E | -30+$\Delta d_E$ = 70 | (5, 6) |
| 6 | 6 | 70 | NE | 70+$\Delta d_{NE}$ = 30 | (6, 6) |
| 7 | 7 | | | | |

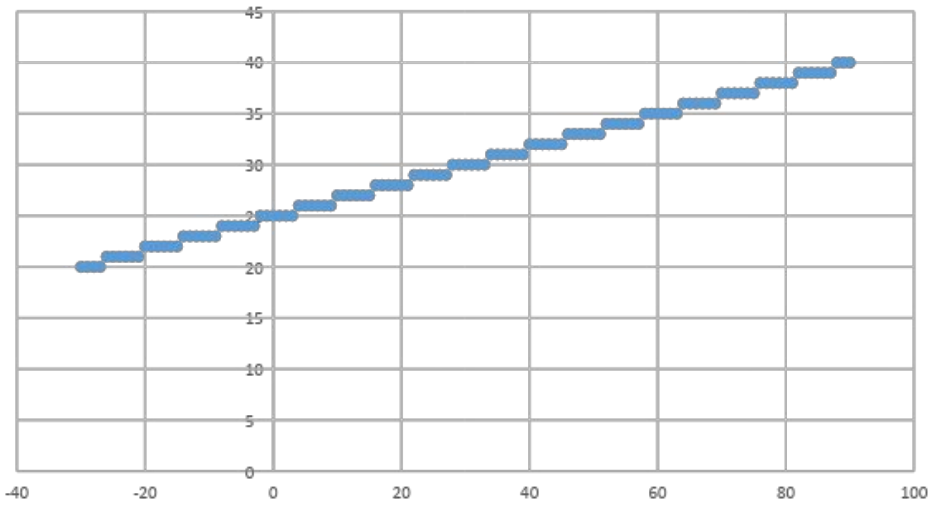
# Midpoint Line Drawing Algorithm

```
Midpoint (x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;
    D = 2*dy – dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++ ;
        if (D>0){
        y++;
        D = D+ ΔNE ;
        }
        else{
        D = D+ ΔE ;
        }
    }
}
```

# (-30, 20) to (90, 40)

dx= 90 + 30 = 120 ; dy = 40- 20 = 20;

D = 2*20 – 120 = -80; ΔNE = 2*(20-120) = -200 ; ΔE = 2*20 = 40;



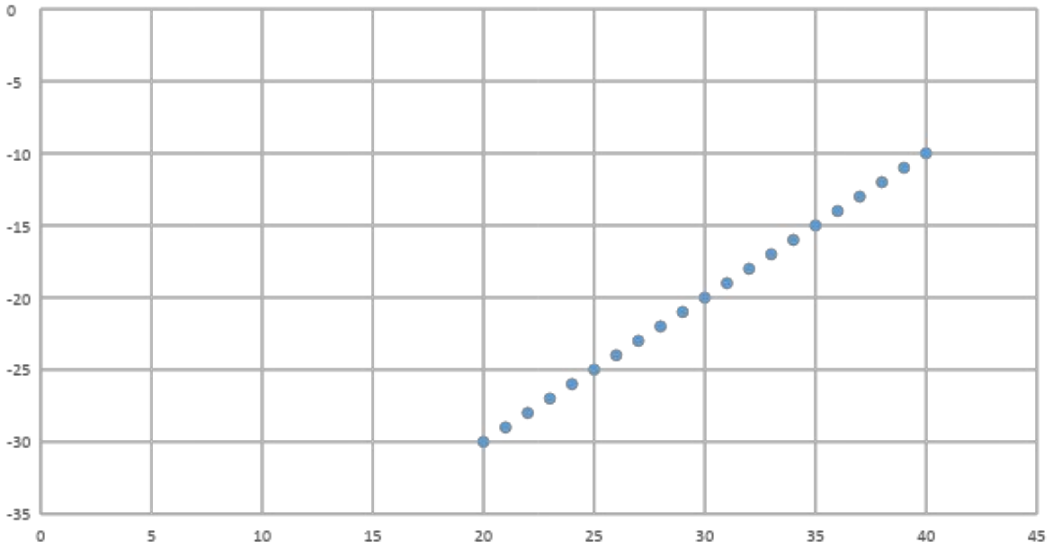

$$m = \frac{20}{120} = 0.167 < 1$$

| X | Y | D |
|---|---|---|
| -30 | 20 | -80 |
| -29 | 20 | -40 |
| -28 | 20 | 0 |
| -27 | 20 | 40 |
| -26 | 21 | -160 |
| -25 | 21 | -120 |
| -24 | 21 | -80 |
| -23 | 21 | -40 |
| -22 | 21 | 0 |
| -21 | 21 | 40 |
| -20 | 22 | -160 |
| -19 | 22 | -120 |
| -18 | 22 | -80 |
| -17 | 22 | -40 |
| -16 | 22 | 0 |
| -15 | 22 | 40 |
| -14 | 23 | -160 |
| -13 | 23 | -120 |
| -12 | 23 | -80 |
| -11 | 23 | -40 |

# (20, -30) to (40, 90)

dy= 90 + 30 = 120 ; dx = 40- 20 = 20;

D = 2*120 – 20 = 220; ΔNE = 2*(120-20) = 200 ; ΔE = 2*120 = 240



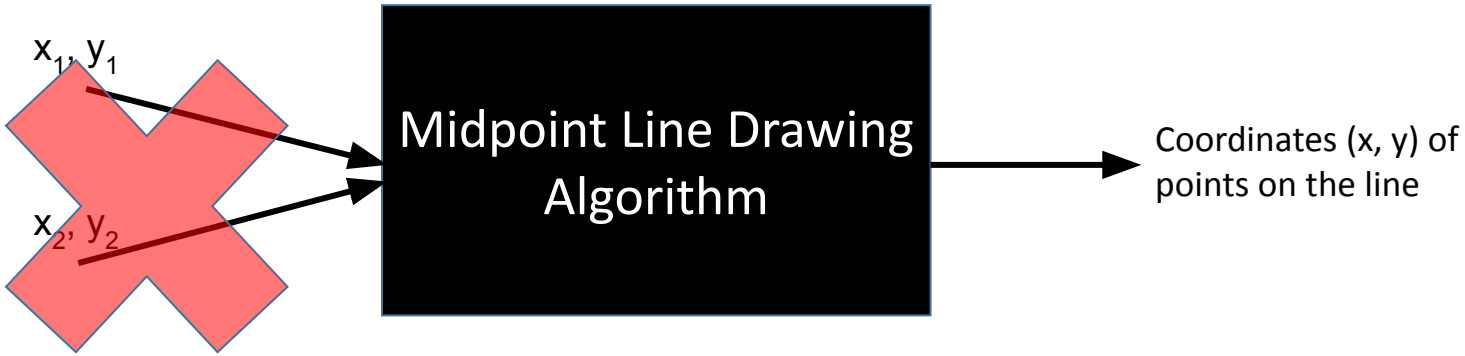

| X | Y | D |
|---|---|---|
| 20 | -30 | 220 |
| 21 | -29 | 420 |
| 22 | -28 | 620 |
| 23 | -27 | 820 |
| 24 | -26 | 1020 |
| 25 | -25 | 1220 |
| 26 | -24 | 1420 |
| 27 | -23 | 1620 |
| 28 | -22 | 1820 |
| 29 | -21 | 2020 |
| 30 | -20 | 2220 |
| 31 | -19 | 2420 |
| 32 | -18 | 2620 |
| 33 | -17 | 2820 |
| 34 | -16 | 3020 |
| 35 | -15 | 3220 |
| 36 | -14 | 3420 |
| 37 | -13 | 3620 |
| 38 | -12 | 3820 |

$$m = \frac{120}{20} = 6 > 1$$

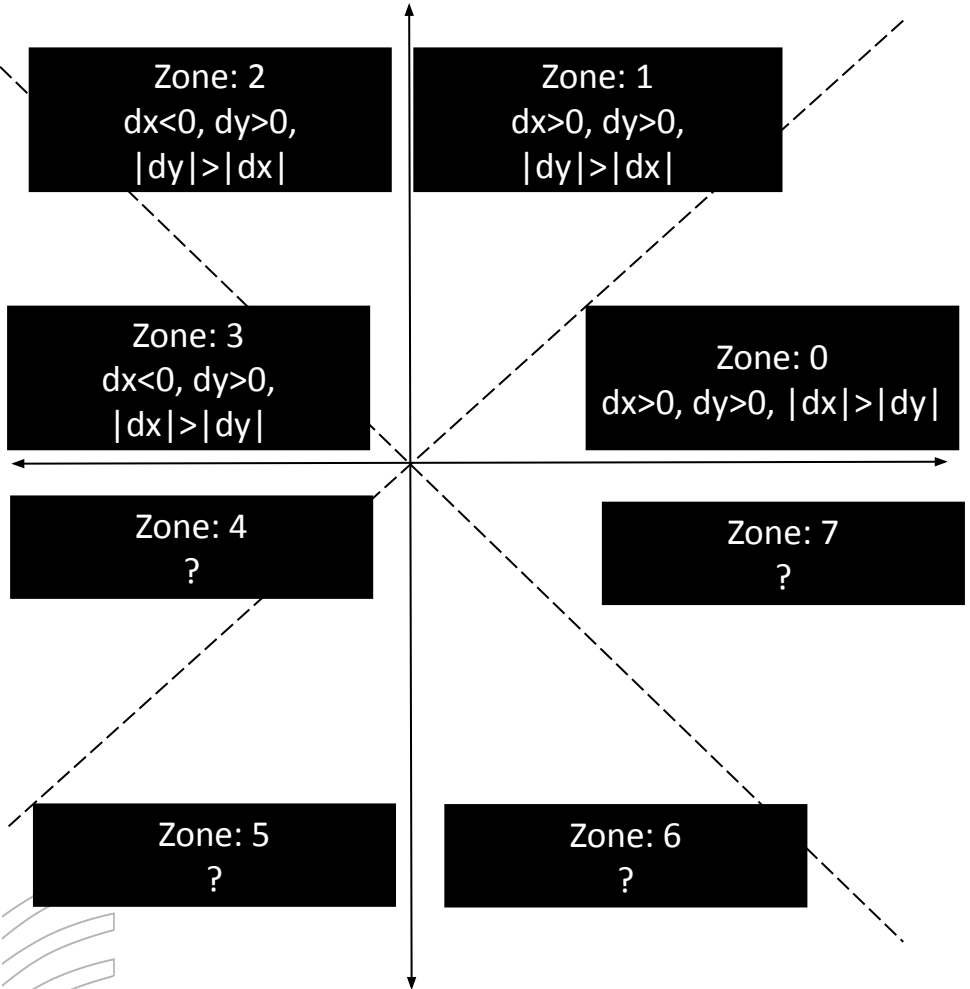
# (30, -20) to (-90, 40)

- 
- If we start from (30, -20), then we need to decrement x to reach (-90, 40)
- If we start from (-90, 40), x will be incremented to reach (30, -20) but y needs to be decremented!
- $m = \frac{60}{-120} = -0.5 < 0$

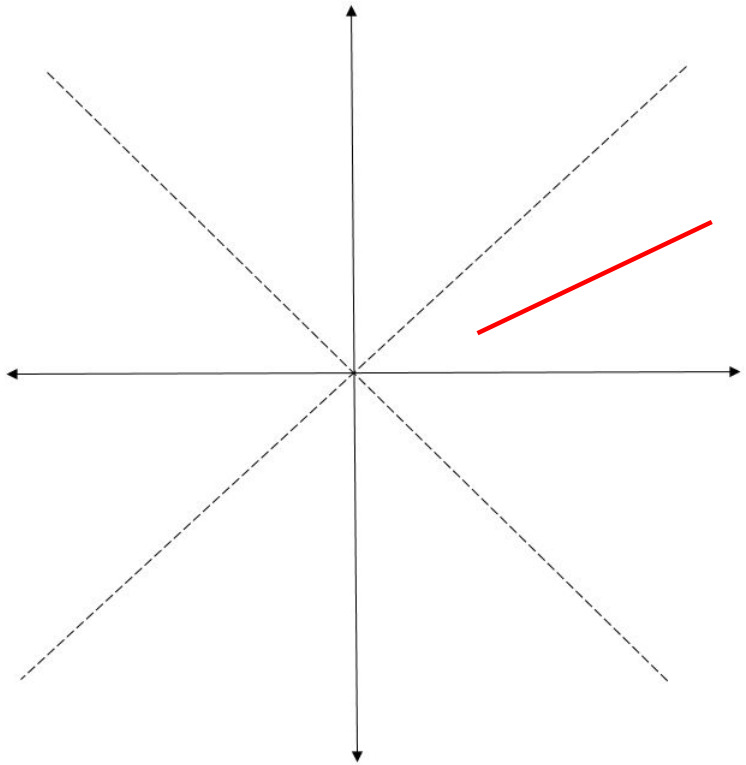
```
Midpoint (x1, y1, x2, y2){
    dx= x2 – x1 ; dy = y2 – y1 ;
    D = 2*dy – dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++ ;
        if (D>0){
            y++;
            D = D+ ΔNE ;
        }
        else{
            D = D+ ΔE ;
        }
    }
}
```

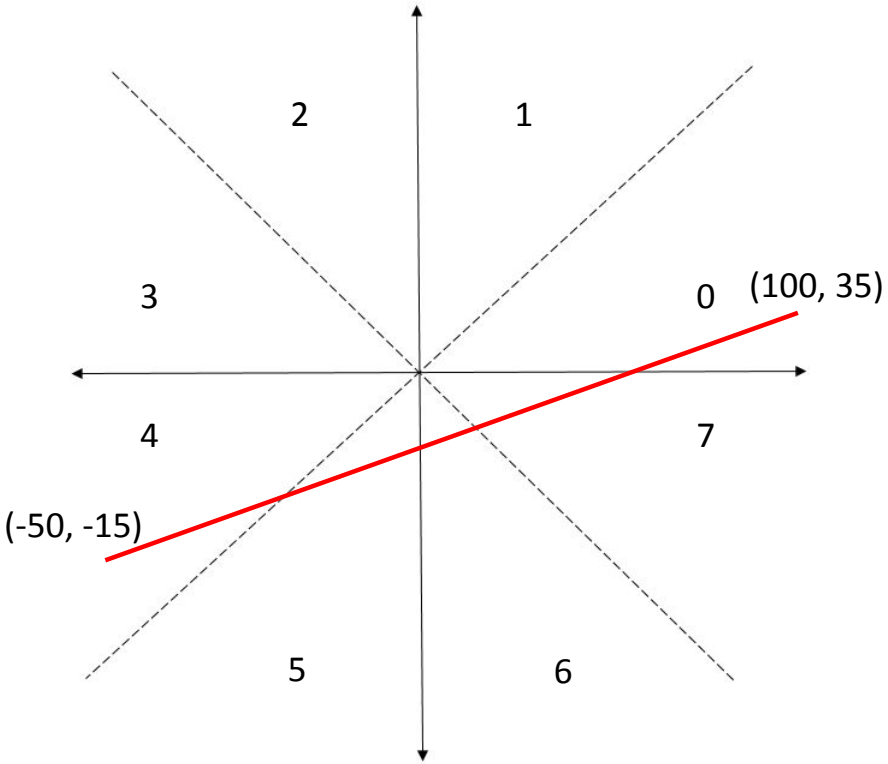

$x_1, y_1$
$x_2, y_2$
Midpoint Line Drawing Algorithm
Coordinates (x, y) of points on the line

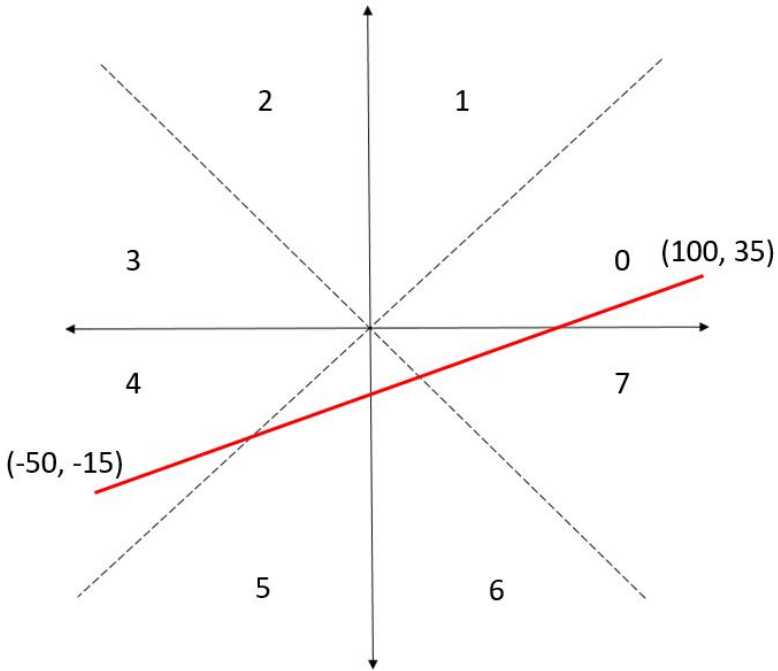
# Eight Way Symmetry




```
FindZone(x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;

    if(|dx| > |dy|){
        if(dx>0 && dy>0) zone = 0;
        else if(dx<0 && dy>0) zone =3;
        else if (? ?) zone = ? ;
        else if (? ?) zone = ?
    }

    else{
        if(dx>0 && dy>0) zone = 1;
        else if(dx<0 && dy>0) zone =2;
        else if (? ?) zone = ? ;
        else if (? ?) zone = ?
    }

}
```




2
1
3
0
(100, 35)
4
7
(-50, -15)
5
6




$dx = 100 + 50 = 150 > 0$

$dy = 35 + 15 = 50 > 0$

$|dx| > |dy|$

Zone = 0

# How do we utilize the zones?

Input $(x_1, y_1)$ to $(x_2, y_2)$ for a line of Zone *M*, where *M* ={0, 1, …, 7}

FindZone

Convert the coordinates of a line in Zone *M* into the coordinates of a line in Zone 0

?

Use the existing midpoint line drawing algorithm for Zone 0

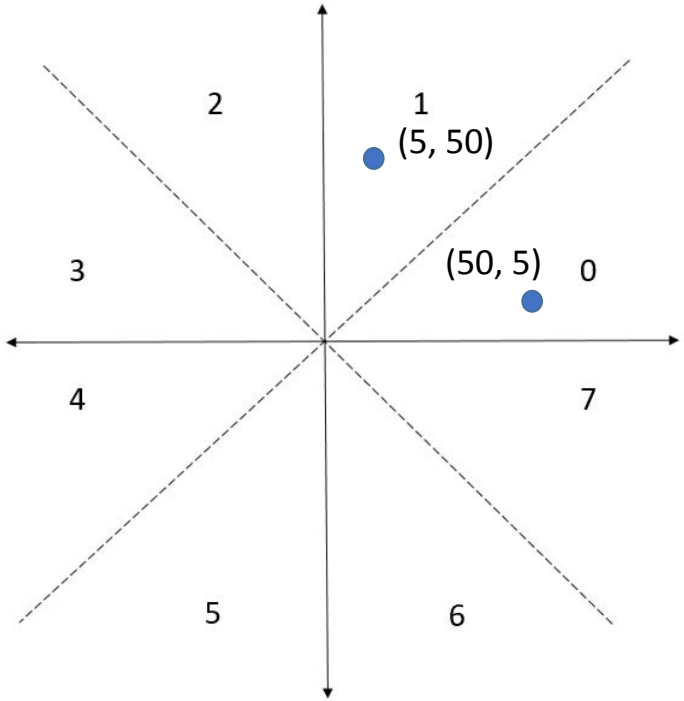
MidPoint

Convert the points (x, y) back to original Zone *M*

?

```
Midpoint (x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;
    D = 2*dy – dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++ ;
        if (D>0){
            y++;
            D = D+ ΔNE ;
        }
        else{
            D = D+ ΔE ;
        }
    }
}
```

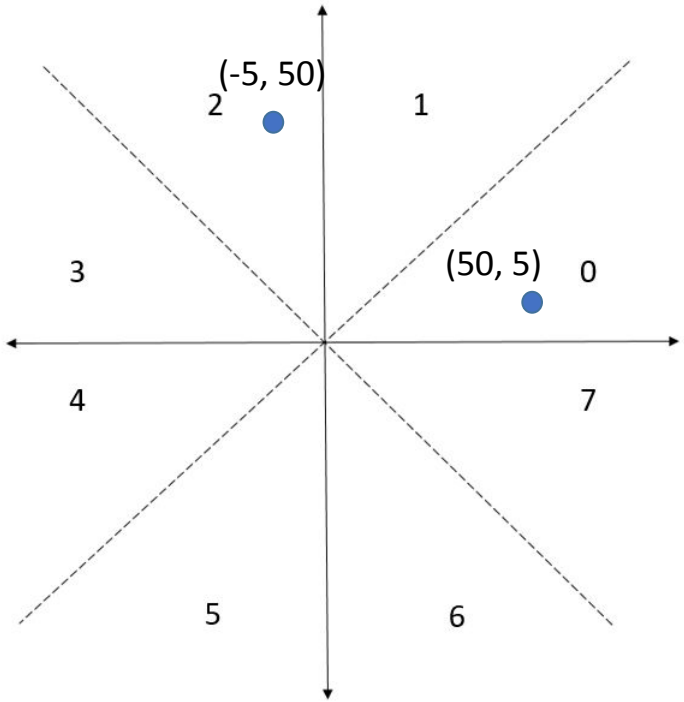
# Convert the coordinates of Zone *M* into the coordinates of Zone 0
# Zone 1 ⟶ Zone 0




Coordinates in Zone 1: ( X, Y ) becomes (Y , X) in Zone 0

```
ConvertToZone0 (X, Y, zone){

    if (zone == 1){
       x = Y, y = X
    }
    return (x, y)

}
```

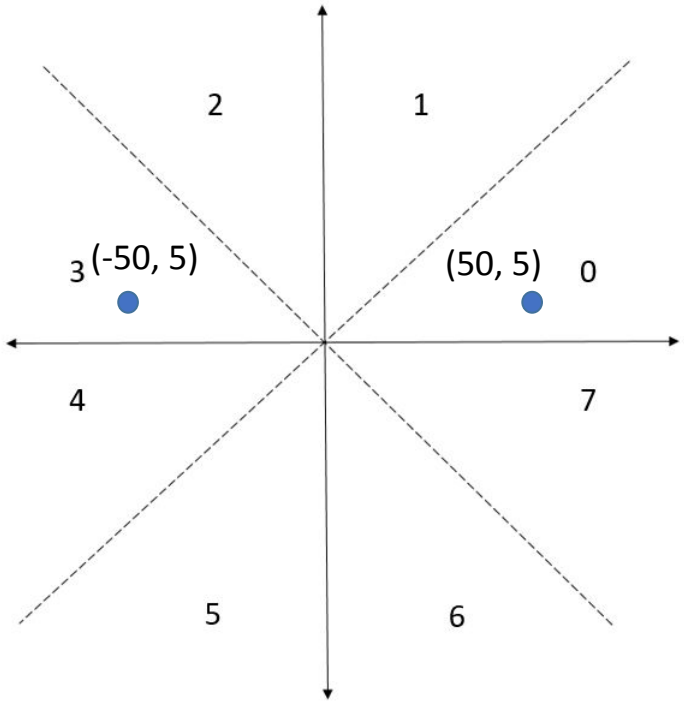
# Convert the coordinates of Zone *M* into the coordinates of Zone 0
# Zone 2 → Zone 0




Coordinates in Zone 2: ( X, Y ) becomes (Y , - X) in Zone 0

```
ConvertToZone0 (X, Y, zone){

    if (zone == 1){
      x = Y, y = X
    }
    else if (zone ==2){
      x = Y, y = -X
    }
    return (x, y)

}
```

# Convert the coordinates of Zone *M* into the coordinates of Zone 0
# Zone 3 ⟶ Zone 0




Coordinates in Zone 3: ( X, Y ) becomes (-X, Y) in Zone 0

```
ConvertToZone0 (X, Y, zone){

    if (zone == 1){
      x = Y, y = X
    }
    else if (zone ==2){
      x = Y, y = -X
    }
    else if (zone ==3){
      x = -X , y = Y
    }
    ….
    return (x, y)

}
```

DIY for zone 4, 5, 6, 7

Input $(x_1, y_1)$ to $(x_2, y_2)$ for a line of Zone *M*, where *M* ={0, 1, …, 7}

FindZone

Convert the coordinates of a line in Zone *M* into the coordinates of a line in Zone 0

ConvertToZone0

Use the existing midpoint line drawing algorithm for Zone 0

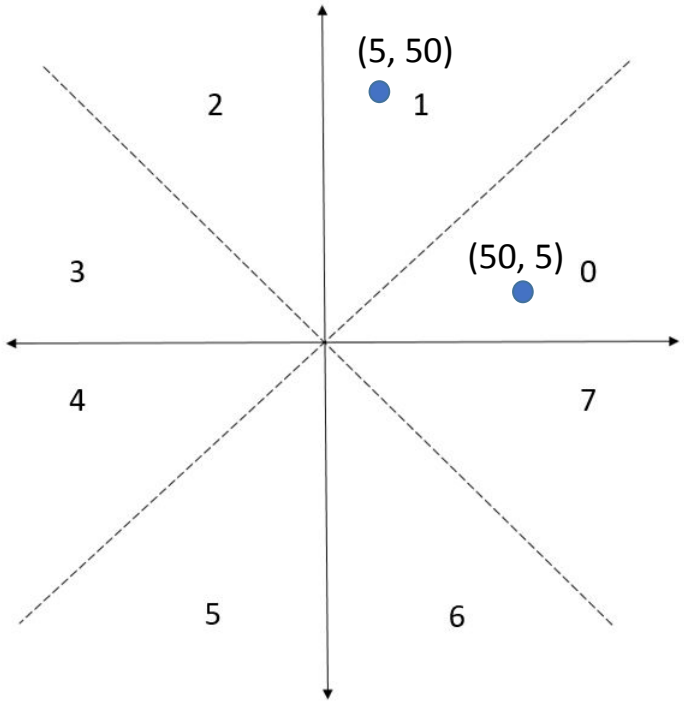
MidPoint

Convert the points (x, y) back to original Zone *M*

?

```
Midpoint (x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;
    D = 2*dy – dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++ ;
        if (D>0){
            y++;
            D = D+ ΔNE ;
        }
        else{
            D = D+ ΔE ;
        }
    }
}
```

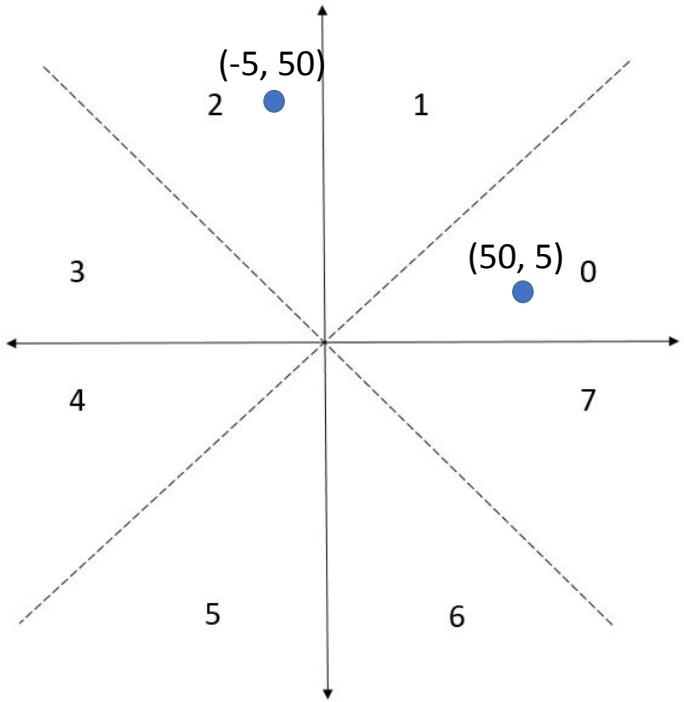
# Go back to original zone M
# Zone 0 ⟶ Zone 1




Coordinates in Zone 0: ( X, Y ) becomes (Y , X) in Zone 1

```
OriginalZone (X, Y, zone){

    if (zone == 1){
       x = Y, y = X
    }
    return (x, y)

}
```

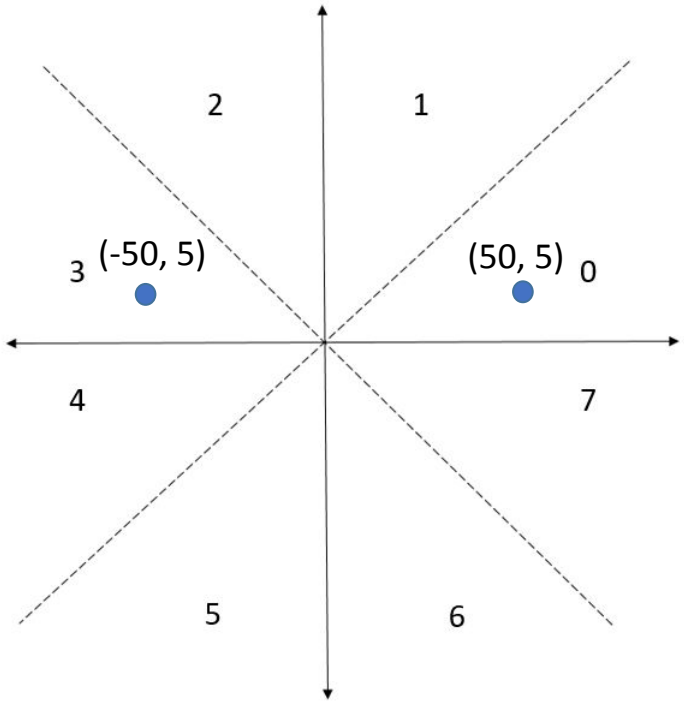
# Go back to original zone M
# Zone 0 ⟶ Zone 2




Coordinates in Zone 0: ( X, Y ) becomes (-Y , X) in Zone 2

```
OriginalZone (X, Y, zone){

    if (zone == 1){
      x = Y, y = X
    }
    else if(zone == 2){
      x = -Y, y = X
    }
    return (x, y)

}
```

# Go back to original zone M
# Zone 0 ⟶ Zone 3




Coordinates in Zone 0: ( X, Y ) becomes (-X , Y) in Zone 3

```
OriginalZone (X, Y, zone){

    if (zone == 1){
       x = Y, y = X
    }
    else if(zone == 2){
       x = -Y, y = X
    }
   else if (zone ==3){
       x = -X , y = Y
    }
    ….
    return (x, y)

}
```

DIY for zone 4, 5, 6, 7

Input $(x_1, y_1)$ to $(x_2, y_2)$ for a line of Zone *M*, where *M* ={0, 1, …, 7}

FindZone

Convert the coordinates of a line in Zone *M* into the coordinates of a line in Zone 0

ConvertToZone0

Use the existing midpoint line drawing algorithm for Zone 0

MidPoint

Convert the points (x, y) back to original Zone *M*

OriginalZone

```
Midpoint (x1, y1, x2, y2){
    dx= x2 - x1 ; dy = y2 - y1 ;
    D = 2*dy – dx; ΔNE = 2*(dy-dx) ; ΔE = 2*dy ;
    x= x1 ; y = y1 ;
    while( x ≤ x2){
        Draw(x, y);
        x++ ;
        if (D>0){
            y++;
            D = D+ ΔNE ;
        }
        else{
            D = D+ ΔE ;
        }
    }
}
```

# Sample Problem and Step by Step Solution

## Find first 7 pixels of the line having endpoints (-10,-20) and (-20, 70)

**Step 1 (Find out the Zone):**

dx = -20 +10 = -10 < 0

dy = 70 + 20 = 90 > 0

|dy| > |dx|,

**So, Zone = 2**

**Step 2 (Convert to Zone 0):**

Zone = 2, so, (x = Y, y = -X)

**Start:** (-10, -20) → **(-20, 10)**

**End:** (-20, 70) →**(70, 20)**

**So, new endpoints in Zone 0: (-20, 10) and (70, 20)**

```
ConvertToZone0 (X, Y, zone){

    if (zone == 1){
      x = Y, y = X
    }
    else if (zone ==2){
      x = Y, y = -X
    }
    else if (zone ==3){
      x = -X , y = Y
    }
    ....
    return (x, y)

}
```

## Step 3 (Apply MPL for Zone 0):

dx' = 70 + 20 = 90,
dy' = 20 - 10 = 10,
D = 2*10 - 90 = -70,
ΔNE = 2*(10-90) = -160,
ΔE = 2*10 = 20

## Step 4 (Convert to Original Zone):

Orange Colored X' and Y' are in Zone 0,
Now, Convert back to Original Zone
Here, Original Zone = 2, So, **(x = -Y, y = X)** ,
Green Colored Part is the actual Pixels (X, Y)

| X' | Y' | D | E/NE | Updating D | Pixels (in Zone 2) X | Pixels (in Zone 2) Y |
|---|---|---|---|---|---|---|
| -20 | 10 | -70 | E | -50 | **-10** | **-20** |
| -19 | 10 | -50 | E | -30 | **-10** | **-19** |
| -18 | 10 | -30 | E | -10 | **-10** | **-18** |
| -17 | 10 | -10 | E | **10** | **-10** | **-17** |
| -16 | 10 | **10** | **NE** | -150 | **-10** | **-16** |
| **-15** | **11** | -150 | E | -130 | **-11** | **-15** |
| -14 | 11 | -130 | E | -110 | **-11** | **-14** |

```
OriginalZone (X, Y, zone){

    if (zone == 1){
        x = Y, y = X
    }
    else if(zone == 2){
        x = –Y, y = X
    }
    else if (zone ==3){
        x = –X , y = Y
    }
    ....
    return (x, y)

}
```

# MPL Simulation Exercise

1. Using the **Slope Independent Line Drawing Algorithm**, find the first 10 missing pixel values starting from: (Show the present value of d as well as Δs at **each stage in a table**)
   a. (20, 10) to (30, 18)
   b. (0, -14) to (-45, 11)
   c. (-1011, -29) to (-5509, 823)
   d. (-18, 11) to (0, -3)
   e. (0, -3) to (-18, 11)
   f. (5, 10) to (-20, 5)
2. Suppose a line segment has the equation $12x - 5y = 0$, and the starting point is at $y = 12$ and the ending point is at $y = 0$. Find the first 6 pixels using the Midpoint Line Drawing Algorithm.
3. A line with the equation, $y = -5x - 20$, starts from where it cuts the y-axis and ends at where it cuts the x-axis. Find the first 8 pixels using the Midpoint Line Drawing Algorithm.

**Note**: *Always **check the zone** first while solving MPL problems. (Check the Sample Solution)*